rogii-wellbore-geology-prediction

14th Place: A Hierarchical U-Net Stack with Seed Averaging

Rank	14
Team	keithtyser
Author	ktyser
Topic ID	733201
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733201
Retrieved with Kaggle CLI on 2026-08-06.

I competed solo and finished 14th with a private score of 6.329, from 80th on the public board.

This writeup covers two models. The one I submitted, and a simpler one I built three days earlier that scored better on the private set.

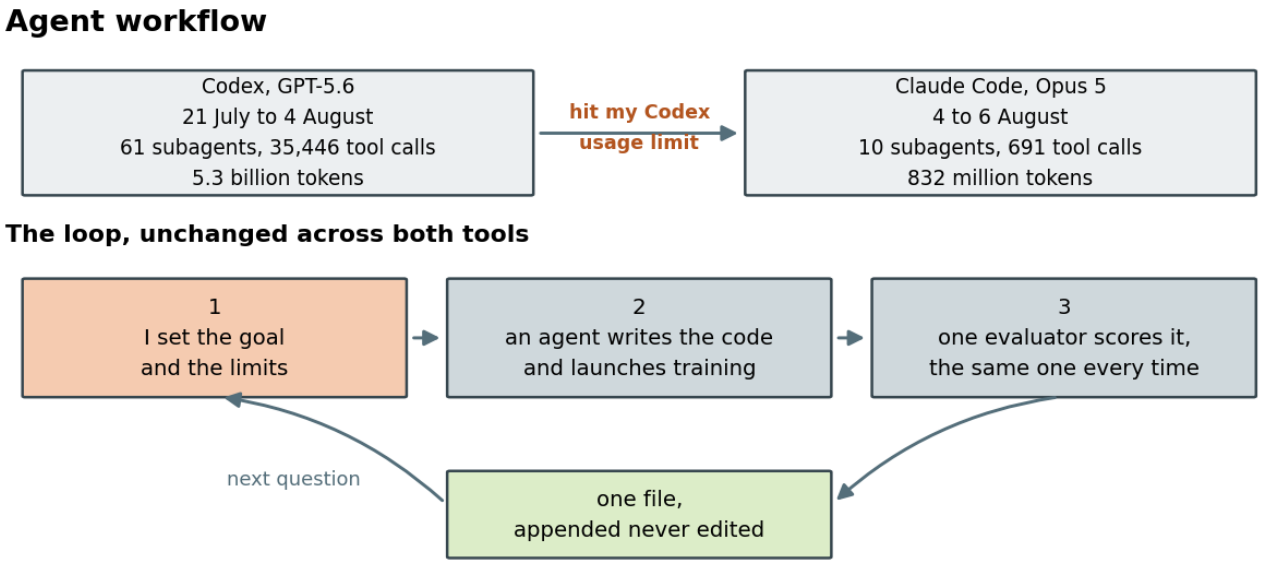
Code

- Training code: https://github.com/keithtyser/rogii-wellbore-geology-solution
- Final submission notebook: https://www.kaggle.com/code/keithtyser/rogii-construction-a-v1-submit?scriptVersionId=340272535

The notebook runs inference from frozen checkpoints. The repository holds the code that produced them, and covers all thirteen of my submissions rather than only the two I selected.

How I worked

I competed solo. I used coding agents to write the code, launch the training, watch the runs, and draft this writeup. I made the calls about what to try and what to ship.



I ran codex with GPT-5.6 from July 21st to August 4th, when I hit my usage limit. I then moved to claude code with Opus 5 for the last two days. The tools changed, but my loop/workflow did not.

One orchestrator held the plan and the constraints. When something needed real work it spawned a subagent with a fresh context, one question, and no history to read. The subagent answered, then the orchestrator wrote the answer to an append-only ledger and moved on.

Agent ledger entries

The agents logged 1,338 entries. 694 report an RMSE, but 87 of those are mid-training readings rather than results: a score logged at epoch 12, then 13, then 20, on a run that had not finished. Two of those chains run 13

entries long. That leaves 607 finished experiments. The rest of the ledger is audits, status notes, packaging steps and infrastructure.

By subject, the 607 are:

- 302 neural network training runs
- 98 on stacking, blending and gating
- 50 on alignment and cross-well geometry
- 48 on gradient boosting and tabular models
- 25 on data and diagnostics
- 24 on packaging and submission
- 60 that fit none of those labels: particle filters, viterbi beam search, spline cost encoders, dynamic-programming slope searches, signed-distance-field CNNs

By outcome, I kept 74 and rejected 237. Another 135 finished without changing anything.

The numbers behind it

The codex phase logged 543 messages from me and 5,411 back, spawned 61 subagents, made 35,446 tool calls of which 27,871 were shell commands, and compacted its own context 314 times. It used 5.3 billion tokens, most of them cached reads of a context that kept being re-sent.

The claude phase ran 36 hours: 10 subagents, 691 tool calls, 832 million tokens.

Hardware was two RTX PRO 6000 Blackwell cards, plus kaggle's own GPUs to check every submission before it counted.

What it is good for

The first gain is speed. An experiment that would have cost me an evening to write, launch and score takes an agent a few minutes. Over sixteen days that changes what is worth trying at all, and I tried things I would otherwise have talked myself out of. Simply put, there's no way I would've been able to run 607 experiments in 16 days without the help of coding agents.

The second gain is that it keeps working when I stop. The agent watched jobs finish, pulled the results, restarted what died, and took the next item off a queue. That ran while I slept and while I was at work.

I spent most of the last few days of the competition on vacation. The remote control in codex and claude code let me read the state from my phone, ask a question, and point the next run somewhere. The work carried on between those check-ins, and the seed farm that produced my final model ran almost entirely while I was away.

What I have not solved is letting an agent hill-climb on its own. Left to pick its own next step it plateaus quickly, or settles into a local minimum and polishes it. Every real change of direction in this competition came from me. I am still working on finding a good workflow for letting agents improve autonomously.

What made it work

One evaluator, used by everything. Every run scored on the same five folds, the same 773 wells, the same 3,783,989 rows. A result from day two stayed comparable with a result from day fourteen. Without that, an agent running hundreds of experiments produces noise with numbers attached.

An append-only record. The ledger was written, never edited. When I asked on the last day what my best CV was, the answer came from a file rather than from anyone's memory of it.

A promotion gate written down in advance: gain at least 0.01 RMSE, with a well-level cluster bootstrap giving P(gain > 0) of 0.9 or better. It rejected 237 of the 607 experiments. The rule was worth more than any model in this writeup, and I overrode it more than once.

Fresh context per question. A subagent that only ever sees one question cannot talk itself into a result it saw earlier.

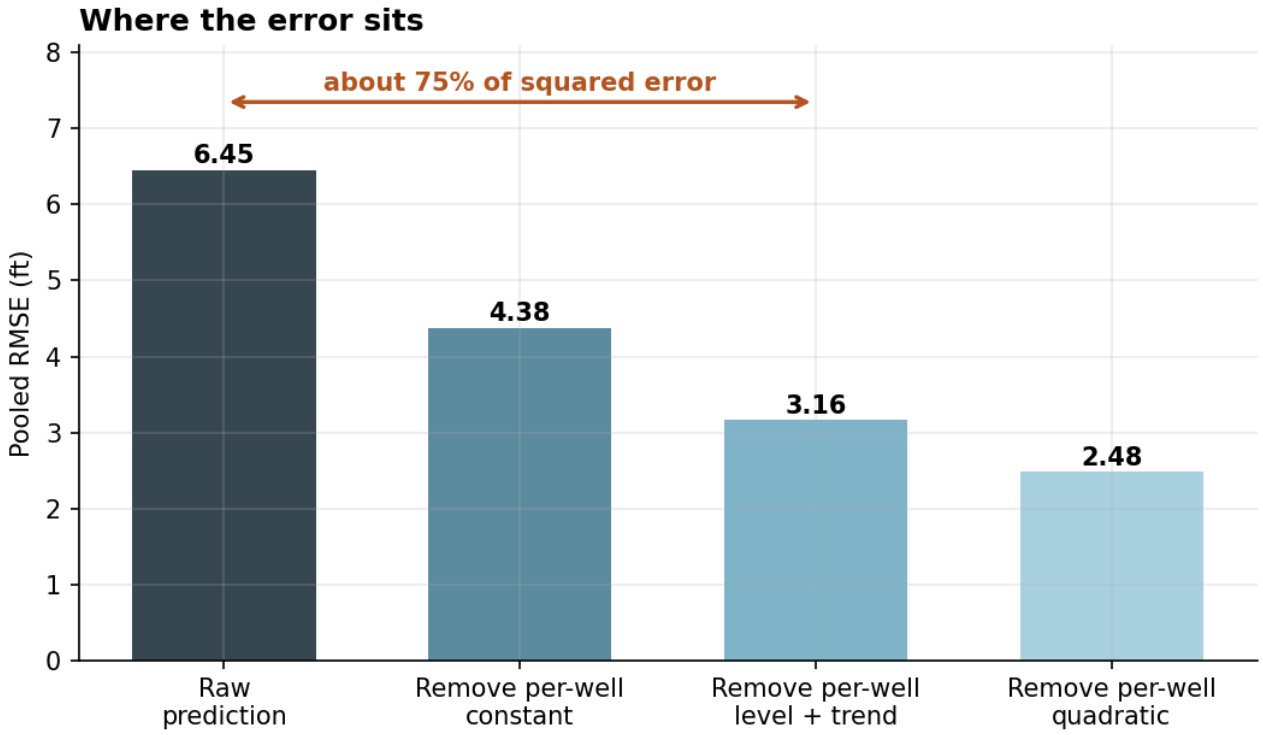
Credit

I started from Tucker Arrants' public work. The U-Net treatment of a well, the synthetic-well pretraining, the decode by expected value over a depth axis: those ideas are his, shared on the forums while the competition ran. Read his writeup. It is better than mine on architecture.

I also used the identity that circulated in the discussions. `TVT + Z - ANCC` stays constant within a well. In my data it holds to about 0.03 ft.

Where the error sits

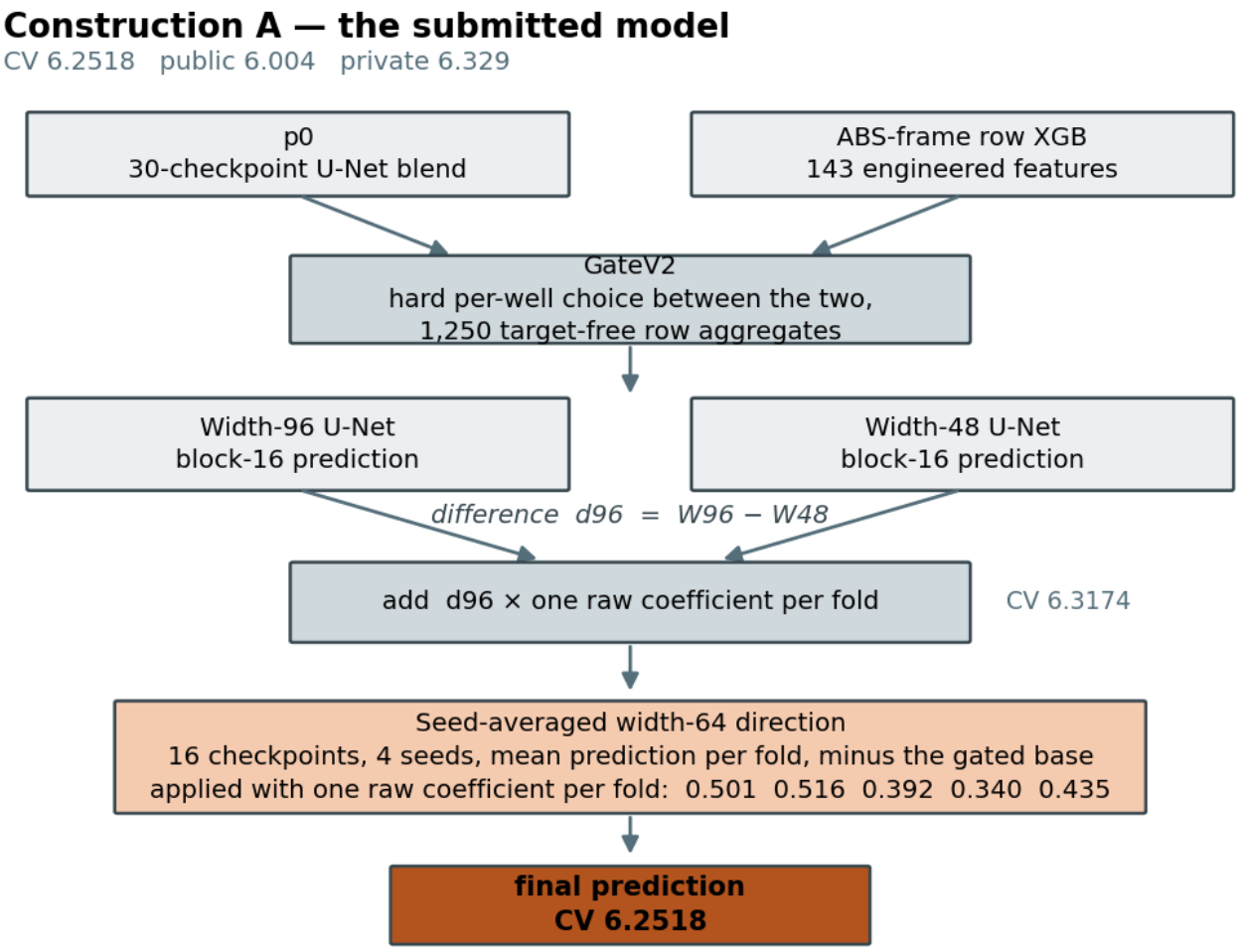
I ran this audit early and it shaped what I built. Take the predictions, then remove a fitted per-well constant, then a per-well line, then a per-well quadratic, and see what is left.



Removing one number per well takes 6.45 to 4.38. Removing a line per well takes it to 3.16. The shape along the well is close to solved. The height of the well in the section is not, and it accounts for about 75% of squared error.

So every model below is a base prediction plus a correction aimed at that offset.

The submitted model



Four stages.

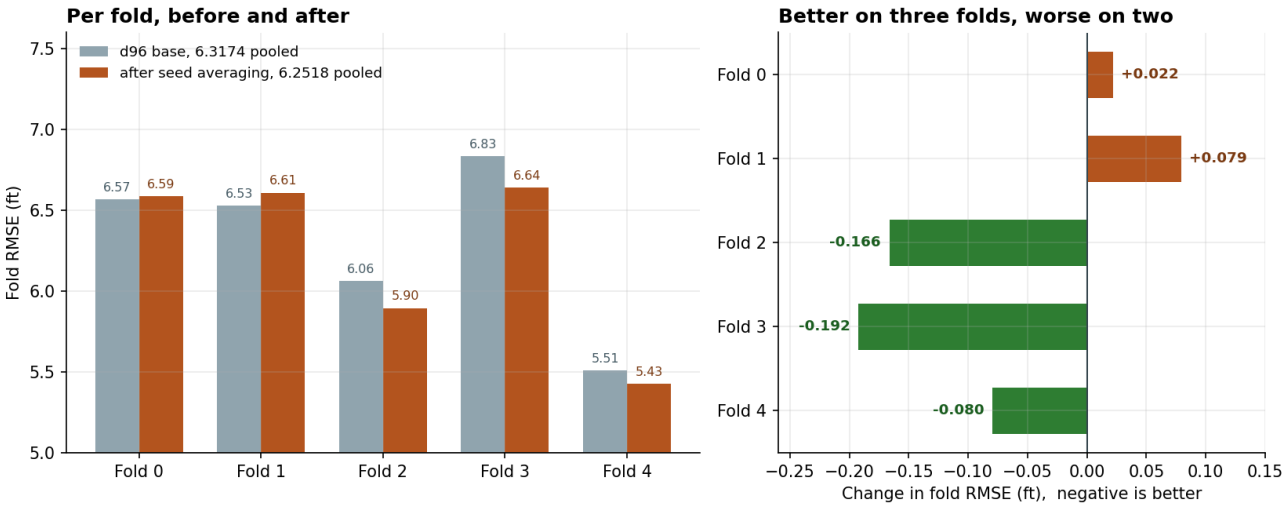
Two base predictions. `p0` is a 30-checkpoint U-Net blend. The second is gradient-boosted trees on 143 engineered features in an absolute reference frame.

GateV2 makes a hard per-well choice between those two. It reads 1,250 aggregates of the row features, none of which touch the target, and picks one model per well. On its own this scored 6.3358.

For the d96 direction I trained the same U-Net at width 96 and width 48, took the difference of their block-16 predictions, and added it back with one raw coefficient per fold. The difference between two widths carries scale information that neither prediction carries alone. Raw width-96 predictions were worse than the base and I rejected them twice; the difference helped. This gave 6.3174, and I submitted it as my second selection.

For the seed-averaged direction I trained the width-64 U-Net under four seeds, averaged the predictions within each fold, subtracted the gated base, and added the result with one coefficient per fold. Sixteen checkpoints in total. Final CV 6.2518.

The per-fold coefficients were 0.501, 0.516, 0.392, 0.340 and 0.435. They are applied to the raw direction without rescaling. In the repository, `train/train_w64_seeds.sh` trains the sixteen checkpoints and `stack/fit_seed_direction.py` fits the coefficients.



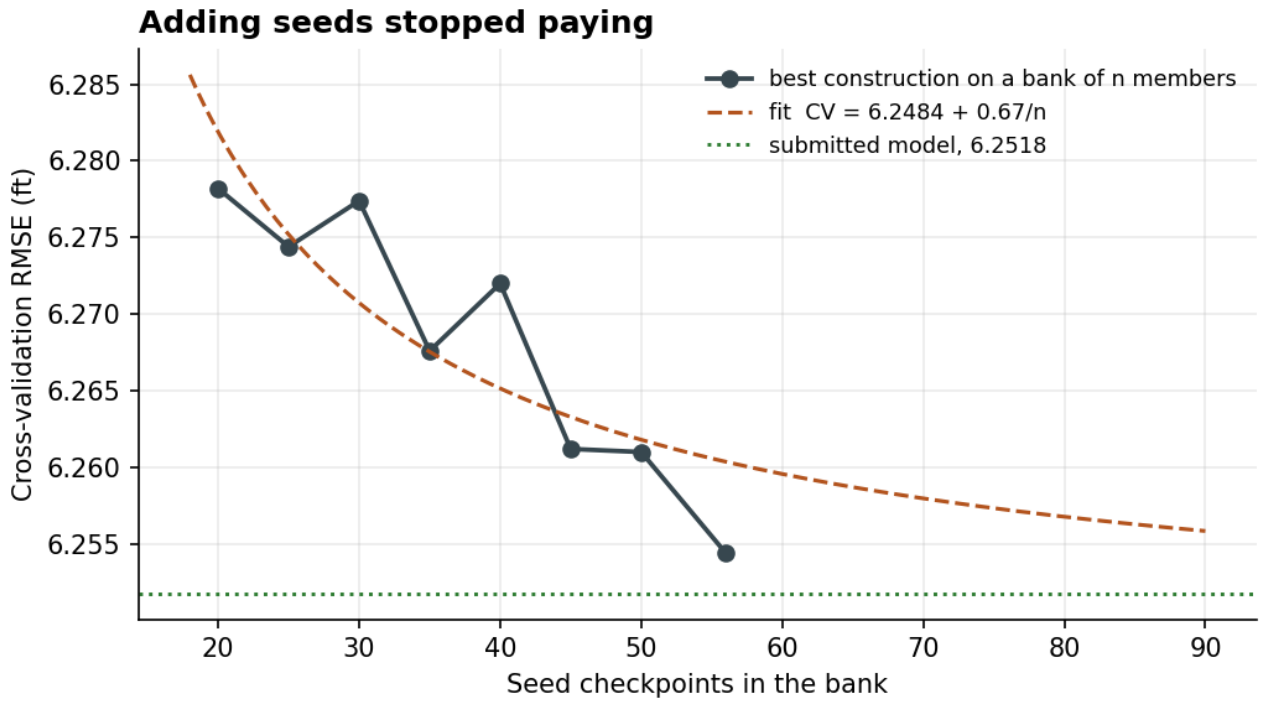
A single seed as a direction was worth about −0.004. The average of several was worth +0.030, and the sixteen-checkpoint roster +0.066. Averaging cuts the variance between seeds, and what survives is the part they agree on.

This was the only late change that moved the public board, from 6.102 to 6.004. On private it moved 6.341 to 6.329.

I set one rule for shipping: gain at least 0.01 CV, with a well-level cluster bootstrap giving P(gain > 0) of 0.9 or better. Seed averaging scored 0.936. Most of my other late candidates scored 0.6 to 0.8, and I shipped several of them anyway.

Where seed averaging stopped

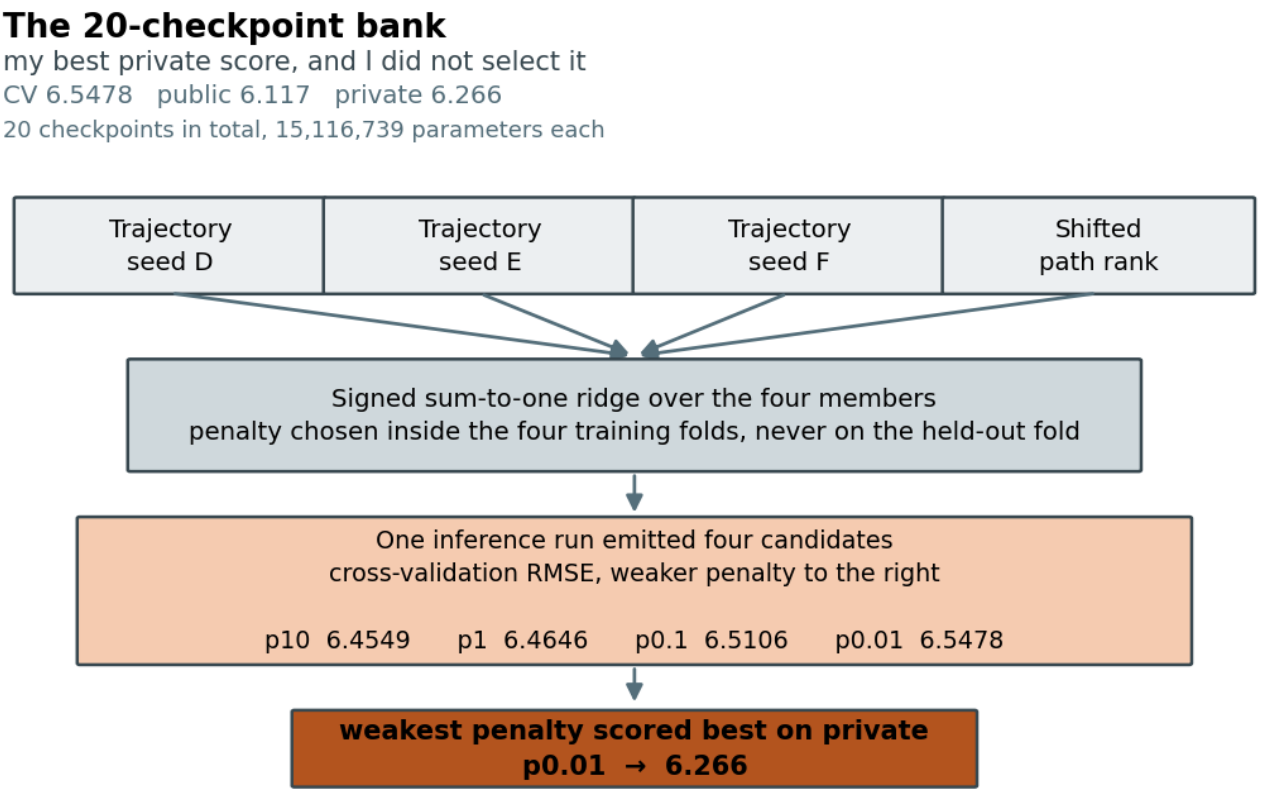
After the deadline for useful training I kept adding seeds, up to 58 checkpoints, and scored the bank three times.



The curve flattens. Fitting CV = a + b/n gives a = 6.2484, so even an unlimited bank lands close to what sixteen checkpoints already reached. Fold 3 is why. Eleven separate width-64 runs on that fold produced a best of 6.957, while the two seeds in my frozen roster reached 6.641 together. I could not reproduce that pair by adding depth.

The model that scored best on private

Three days before the end I had something simpler.



Four members: three U-Nets trained under different trajectory seeds, plus a shifted-path-rank variant. Twenty checkpoints, 15,116,739 parameters each. A signed sum-to-one ridge combined them, with the penalty chosen inside the four training folds and never on the held-out fold.

I shipped four penalties from one inference run. The strongest penalty gave the best CV at 6.4549. The weakest gave the worst CV at 6.5478, and the best private score of anything I submitted, 6.266. The order on private ran opposite to the order on CV, across all four.

How I validated

This is the part of my setup I would keep unchanged.

One split, fixed early and never touched again. Five folds, grouped by whole well, so no well ever appears in both training and validation. 773 wells and 3,783,989 rows, every row scored. The score is pooled row RMSE across all folds at once, not the mean of five fold scores, which stops a small fold from swinging the number.

Anything fitted on out-of-fold predictions was fitted on four folds and applied to the fifth. Ridge penalties, gate thresholds, per-fold coefficients, all of it.

For deciding whether a gain was real I used a cluster bootstrap: resample wells with replacement, 3,000 times, recompute the difference, and read off the fraction of resamples where the change helped. Wells rather than rows, because rows inside a well are heavily correlated and bootstrapping them would report false confidence.

That gave a CV that tracked the public board closely. Across my ten submissions, CV against public score is Pearson +0.83 and Spearman +0.81. The offset is stable too: CV sat 0.34 higher than public on average, with a standard deviation of 0.086 across a CV range of 0.43. So a CV improvement usually showed up on the board, and I could tell roughly what public score a CV number would produce before submitting.

Choosing the two submissions

The first pick was easy. Construction A had the best CV and the best public score, and nothing else was close on either.

The second pick was the real decision, and I got it wrong. My instinct was to hedge with something structurally different, so the two picks would not fail the same way. The obvious candidate was the xgboost model, a different family entirely, sitting at public 6.094.

So I measured it. On aligned out-of-fold predictions across all 3,783,989 rows, the xgboost model correlates with Construction A at 0.982, and with the hierarchical d96 stack at 0.994. Trees against convolutions turned out to be a distinction without a difference. Both models anchor on the same incumbent prediction and add a small correction, so their errors land in the same places.

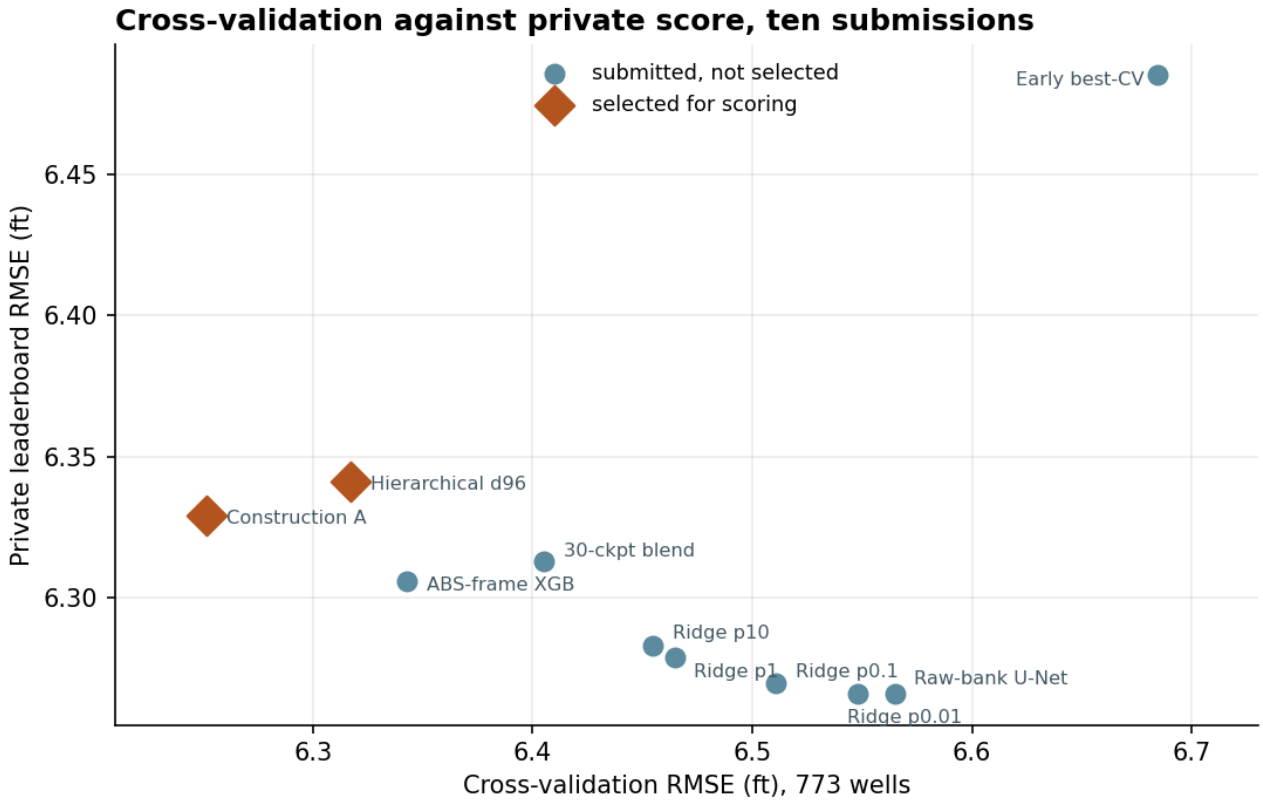
With diversity worth almost nothing, I picked on expected score instead. A well-cluster bootstrap over 148-well draws, taking the better of each pair the way the leaderboard does, put the d96 pair at 6.213 and the xgboost pair at 6.214. The d96 pair was slightly better in the tail as well.

The d96 stack had a second argument for it. It is the exact base layer inside Construction A, so if the seed-averaged direction failed to transfer, the second pick removed precisely that one component and kept the rest. A surgical fallback rather than a different bet.

I ignored the public board here, and I would again. The nine submissions between 6.093 and 6.119 sit inside the noise of a 52-well split, so ranking them by public score seemed like a trap.

What none of that caught is that both picks came from the top of my CV ranking, and my CV ranking was inverted on the private set. Two picks from the same end of a list that was pointing the wrong way is one pick, not two.

Final scores



Submission	CV	Public	Private
Construction A (selected)	6.2518	6.004	6.329
Hierarchical d96 (selected)	6.3174	6.102	6.341
ABS-frame row XGB	6.3429	6.094	6.306
30-checkpoint U-Net blend	6.4052	6.110	6.313
Deployable ridge p10	6.4549	6.093	6.283
Deployable ridge p1	6.4646	6.096	6.279
Deployable ridge p0.1	6.5106	6.108	6.270
Deployable ridge p0.01	6.5478	6.117	6.266
Deployable raw-bank U-Net	6.5648	6.119	6.266
Early best-CV U-Net	6.6846	6.258	6.485

Numbers

- 1,338 log entries over 16 days, of which 607 are finished experiments
- 5.4 billion tokens burned
- 773 wells, 3,783,989 rows, fixed 5-fold whole-well grouped split, pooled row RMSE

- CV 7.51 to 6.25, public 6.554 to 6.004, private 6.329
- Compute: 2x RTX Pro 6000s

Thanks

To Tucker (@tuckerarrants), for the architecture and for answering questions on the forum all competition. To the people who shared the structural identity and the early baselines. And to ROGII and the competition organizers, for a super fun competition.